

```
/**
 * server/services/longform-prep-pipeline.ts
 *
 * Long-form Pre-Publication Pipeline
 *
 * Identical philosophy to shorts-prep-pipeline: ALL AI work happens here,
 * BEFORE any YouTube API call. When quota resets, the publisher just reads
 * ready rows and uploads – zero AI at upload time.
 *
 * Pipeline per video:
 * 1. Load video + back catalog data from DB
 * 2. AI: strategic scoring (should this video publish now vs defer?)
 * 3. AI: generate YouTube title (≤100 chars, search-optimized)
 * 4. AI: generate full SEO description with keyword sections + timestamps
 * 5. AI: generate tag set (≤500 chars, ranked by search volume)
 * 6. AI: generate chapter markers if clip has structure
 * 7. AI: generate thumbnail concept (composition + text + color direction)
 * 8. Write all metadata to DB → status: 'ready_to_upload'
 */
```

```
import { callOpenAI } from '../lib/openai.js';
import { callClaude } from '../lib/claude.js';
import { createLogger } from '../lib/logger.js';
import { setJitteredInterval } from '../lib/timer-utils.js';
import { assertTierCapacity } from '../lib/ai-semaphore.js';
import { storage } from '../storage.js';
```

```
const log = createLogger('longform-prep-pipeline');
```

```
// — Types —
```

```
interface VideoRecord {
  id: number;
  userId: string;
  youtubeVideoId: string | null;
  title: string;
  description: string | null;
  tags: string[];
  durationSec: number;
  viewCount: number;
  likeCount: number;
  gameName: string;
  channelName: string;
  filePath: string;
  publishedAt: Date | null;
  score: number | null;
}
```

```

interface ChapterMarker {
  timestampSec: number;
  label: string;
}

export interface LongformReadyPayload {
  videoId: number;
  // YouTube metadata – fully pre-computed
  title: string; // ≤100 chars
  description: string; // full SEO description ≤5000 chars
  tags: string[]; // ≤500 chars total
  categoryId: string; // '20' = Gaming
  defaultLanguage: string;
  chapters: ChapterMarker[];
  // Thumbnail
  thumbnailConcept: ThumbnailConcept;
  thumbnailFilePath: string | null;
  // Pipeline state
  status: 'ready_to_upload';
  preparedAt: Date;
}

interface ThumbnailConcept {
  composition: string;
  textOverlay: string;
  colorGrade: string;
  focalElement: string;
  mood: string;
  avoidElements: string;
}

// — Main prep function —————

export async function prepareLongformForUpload(video: VideoRecord):
Promise<LongformReadyPayload> {
  log.info(`[LongformPrepPipeline] Starting prep for video ${video.id} – "${video.title}"`);

  const durationMin = Math.round(video.durationSec / 60);

  // Step 1 – Strategic scoring: should this video publish now?
  assertTierCapacity('longform_pipeline', 'longform-prep');
  const strategyResult = await callOpenAI({
    tier: 'longform_pipeline',
    messages: [
      {
        role: 'system',
        content:
          'You are a YouTube growth strategist for a no-commentary PS5 gaming channel. ' +
          'Score how well a video fits current publishing conditions. ' +
          'Respond ONLY with JSON: { "publishScore": 0-100, "reason": "one sentence" }',
      },
    ],
  });
}

```

```

    },
    {
      role: 'user',
      content:
        `Game: ${video.gameName}\n` +
        `Title: ${video.title}\n` +
        `Duration: ${durationMin} minutes\n` +
        `Historical views: ${video.viewCount}\n` +
        `Historical likes: ${video.likeCount}\n\n` +
        'Score how well this video should perform if published today.',
    },
  ],
  maxTokens: 80,
});

let publishScore = 70;
try {
  const raw = strategyResult.choices[0].message.content?.trim() ?? '{}';
  const parsed = JSON.parse(raw.replace(/``json|``/g, '').trim());
  publishScore = parsed.publishScore ?? 70;
} catch { /* use default */ }

log.info(`[LongformPrepPipeline] Video ${video.id} publish score: ${publishScore}/100`);

// Step 2 – Title generation
assertTierCapacity('longform_pipeline', 'longform-prep');
const titleResult = await callOpenAI({
  tier: 'longform_pipeline',
  messages: [
    {
      role: 'system',
      content:
        'You are a YouTube title writer for a no-commentary PS5 gaming channel. ' +
        'Write a title that: (1) is ≤100 characters, (2) leads with the most ' +
        'searchable keyword for this game + video type, (3) creates curiosity or ' +
        'communicates clear value, (4) no clickbait, no ALL CAPS, no emojis. ' +
        'Respond with ONLY the title.',
    },
    {
      role: 'user',
      content:
        `Game: ${video.gameName}\n` +
        `Original title: ${video.title}\n` +
        `Duration: ${durationMin} minutes\n` +
        `Description preview: ${video.description?.slice(0, 300) ?? 'none'}\n\n` +
        'Write the optimized YouTube title.',
    },
  ],
  maxTokens: 60,
});

```

```

const title = titleResult.choices[0].message.content?.trim().slice(0, 100) ?? video.title;
log.info(`[LongformPrepPipeline] Video ${video.id} title: "${title}"`);

// Step 3 – Full SEO description
assertTierCapacity('longform_pipeline', 'longform-prep');
const descResult = await callOpenAI({
  tier: 'longform_pipeline',
  messages: [
    {
      role: 'system',
      content:
        'You are an SEO specialist for a no-commentary PS5 gaming YouTube channel. ' +
        'Write a full YouTube video description with these sections:\n' +
        '1. Opening hook (1-2 sentences, most searchable keyword in first 125 chars)\n' +
        '2. What the video covers (2-3 sentences)\n' +
        '3. Subscribe + notification CTA\n' +
        '4. Keyword section: "Watch more [Game] gameplay:" followed by 3-4 related search phrases\n' +
        '5. Hashtags: 5-8 relevant tags including game name\n\n' +
        'Total: 400-800 characters. No filler. No fake timestamps unless provided.',
    },
    {
      role: 'user',
      content:
        `Game: ${video.gameName}\n` +
        `Title: ${title}\n` +
        `Duration: ${durationMin} minutes\n` +
        `Channel: ${video.channelName}\n` +
        `Original description: ${video.description?.slice(0, 500) ?? 'none'}\n\n` +
        'Write the full SEO description.',
    },
  ],
  maxTokens: 600,
});

const description = descResult.choices[0].message.content?.trim().slice(0, 5000) ?? '';
log.info(`[LongformPrepPipeline] Video ${video.id} description ready (${description.length} chars)`);

// Step 4 – Tag set
assertTierCapacity('longform_pipeline', 'longform-prep');
const tagsResult = await callOpenAI({
  tier: 'longform_pipeline',
  messages: [
    {
      role: 'system',
      content:
        'YouTube SEO specialist. Generate a tag set for a long-form gaming video. ' +
        'Rules: 10-20 tags, mix of broad game/console tags and specific content tags, ' +

```

```

        'total ≤500 chars, ranked best-first by estimated search volume. ' +
        'Respond ONLY with a JSON array of strings.',
    },
    {
        role: 'user',
        content:
            `Game: ${video.gameName}\n` +
            `Title: ${title}\n` +
            `Duration: ${durationMin} minutes\n` +
            `Existing tags: ${video.tags?.slice(0, 10).join(', ')} ?? 'none'\n\n` +
            'Generate the tag array.',
    },
],
maxTokens: 250,
});

let tags: string[] = [];
try {
    const raw = tagsResult.choices[0].message.content?.trim() ?? '[]';
    const parsed = JSON.parse(raw.replace(/```json|```/g, '').trim()) as string[];
    let totalChars = 0;
    tags = parsed.filter(t => { totalChars += t.length + 1; return totalChars <= 500; });
} catch {
    log.warn(`[LongformPrepPipeline] Video ${video.id} tag parse failed – using fallback`);
    tags = [video.gameName, 'Gaming', 'PS5', 'NoCommentary', 'Gameplay'];
}

// Step 5 – Chapter markers (if video is long enough)
let chapters: ChapterMarker[] = [];
if (video.durationSec > 300) {
    assertTierCapacity('longform_pipeline', 'longform-prep');
    const chaptersResult = await callOpenAI({
        tier: 'longform_pipeline',
        messages: [
            {
                role: 'system',
                content:
                    'Generate YouTube chapter markers for a no-commentary gaming video. ' +
                    'Rules: first chapter MUST be "0:00 Intro", space chapters evenly, ' +
                    '4-8 chapters total, labels ≤30 chars each. ' +
                    'Respond ONLY with JSON array: [{ "timestampSec": 0, "label": "Intro" }, ...]',
            },
            {
                role: 'user',
                content:
                    `Game: ${video.gameName}\n` +
                    `Title: ${title}\n` +
                    `Total duration: ${durationMin} minutes (${video.durationSec}s)\n\n` +
                    'Generate chapter markers.',
            },
        ],
    });
}

```

```

    ],
    maxTokens: 200,
  });

  try {
    const raw = chaptersResult.choices[0].message.content?.trim() ?? '[]';
    chapters = JSON.parse(raw.replace(/```json|```/g, '').trim());
  } catch {
    chapters = [{ timestampSec: 0, label: 'Intro' }];
  }
}

// Step 6 – Thumbnail concept (Claude for visual reasoning)
assertTierCapacity('longform_pipeline', 'longform-prep');
const thumbResult = await callClaude({
  tier: 'longform_pipeline',
  messages: [
    {
      role: 'user',
      content:
        'Design a YouTube gaming thumbnail concept for a long-form video.\n\n' +
        `Game: ${video.gameName}\n` +
        `Title: ${title}\n` +
        `Duration: ${durationMin} minutes\n\n` +
        'Output ONLY valid JSON:\n' +
        '{\n' +
        '  "composition": "subject positioning, rule-of-thirds notes",\n' +
        '  "textOverlay": "max 5 words, impact phrase",\n' +
        '  "colorGrade": "shadow and highlight color treatment",\n' +
        '  "focalElement": "single most eye-catching element + frame position",\n' +
        '  "mood": "2-3 emotion words",\n' +
        '  "avoidElements": "UI elements or visual clutter to exclude"\n' +
        '}\n\n' +
        'For PS5 gaming: favor destruction/action in upper frame, ' +
        'strong contrast between foreground subject and background, ' +
        'minimal on-screen HUD, one dominant accent color against desaturated background.',
    },
  ],
  maxTokens: 300,
});

let thumbnailConcept: ThumbnailConcept = {
  composition: 'center subject, enemy threat upper right quadrant',
  textOverlay: 'MUST WATCH GAMEPLAY',
  colorGrade: 'cool blue shadows, warm orange highlights on subject',
  focalElement: 'player character silhouette against explosion, lower left',
  mood: 'epic, intense, awe',
  avoidElements: 'no HUD overlay, no minimap, no health bars',
};

```

```

try {
  const raw = thumbResult.content[0].type === 'text'
    ? thumbResult.content[0].text.trim()
    : '{}';
  thumbnailConcept = { ...thumbnailConcept, ...JSON.parse(raw.replace(/```json|```/g,
'').trim()) };
} catch {
  log.warn(`[LongformPrepPipeline] Video ${video.id} thumbnail parse failed – using
defaults`);
}

// Step 7 – Append chapters to description if present
let finalDescription = description;
if (chapters.length > 0) {
  const chapterText = chapters
    .map(c => {
      const m = Math.floor(c.timestampSec / 60);
      const s = String(c.timestampSec % 60).padStart(2, '0');
      return `${m}:${s} ${c.label}`;
    })
    .join('\n');
  finalDescription = `${description}\n\n${chapterText}`.slice(0, 5000);
}

// Step 8 – Persist to DB
const payload: LongformReadyPayload = {
  videoId:      video.id,
  title,
  description:  finalDescription,
  tags,
  categoryId:   '20',
  defaultLanguage: 'en',
  chapters,
  thumbnailConcept,
  thumbnailFilePath: null,
  status:       'ready_to_upload',
  preparedAt:    new Date(),
};

await storage.upsertLongformReadyPayload(video.id, payload);

log.info(
  `[LongformPrepPipeline]  Video ${video.id} → ready_to_upload | ` +
  `score: ${publishScore} | title: "${title}" | tags: ${tags.length} | chapters:
${chapters.length}`
);

return payload;
}

```

// — Batch runner —

```
export async function runLongformPrepCycle(userId: string): Promise<void> {
  log.info(`[LongformPrepPipeline] Starting prep cycle for user ${userId}`);

  const pendingVideos = await storage.getDownloadedVideosWithoutReadyPayload(userId);
  log.info(`[LongformPrepPipeline] ${pendingVideos.length} videos awaiting prep`);

  for (const video of pendingVideos) {
    try {
      await prepareLongformForUpload(video);
      await new Promise(r => setTimeout(r, 5000)); // 5s between videos
    } catch (err: any) {
      if (err.message?.includes('AI queue full')) {
        log.warn(`[LongformPrepPipeline] Queue full – pausing, will retry next interval`);
        break;
      }
      log.error(`[LongformPrepPipeline] Failed to prep video ${video.id}:`, err);
    }
  }

  log.info(`[LongformPrepPipeline] Prep cycle complete`);
}
```

// — Service lifecycle —

```
let stopPrepCycle: (() => void) | null = null;

export function startLongformPrepPipeline(userId: string): void {
  if (stopPrepCycle) {
    log.warn(`[LongformPrepPipeline] Already running – skipping double-start`);
    return;
  }

  log.info(`[LongformPrepPipeline] Starting – prep interval: ~30 min`);

  stopPrepCycle = setJitteredInterval(
    () => runLongformPrepCycle(userId).catch(err =>
      log.error(`[LongformPrepPipeline] Cycle error:`, err)
    ),
    30 * 60 * 1000, // 30 min base, ±20% jitter
  );
}

export function stopLongformPrepPipeline(): void {
  if (stopPrepCycle) {
    stopPrepCycle();
    stopPrepCycle = null;
    log.info(`[LongformPrepPipeline] Stopped`);
  }
}
```

}

}